

Prepared by: Ryan Stefanic

PSU Email: rts5442@psu.edu

Kaggle Username: Ryan Stefanic

Final Rank in the Kaggle Leaderboard:

10	XinruiLi3		0.90547	6	4d
11	Srikar Lanka	 	0.90384	24	8h
12	JakeCohen1818	 	0.90196	25	1m
 Your Best Entry! Your most recent submission scored 0.90196, which is an improvement of your previous score of 0.89756. Great job! Tweet this					
13	Rishi & Zach (Team 11)	 	0.90099	9	6h
14	Ga Ryoung Lee	 	0.90099	8	2d

Problem Statement:

In this project, we aimed to build a classification model for predicting whether a patient had heart disease or not, using various machine-learning techniques. Below is a detailed description of our approach, covering data pre-processing, feature engineering, model building and selection, hyperparameter tuning, performance evaluation, and lessons learned during the process.

Description:

The first thing we did was preprocess our data. When analyzing the data, we noticed that there were many zeros in the cholesterol column. These zeros were outliers and didn't make

sense compared with the range of the rest of the data. To fix this issue, we replaced the zero values with the median cholesterol value of the other rows. The next thing we observed was that the dataset contained categorical variables that included both text and numerical data. To process the text values, we used one-hot encoding to turn the text into binary vectors, which allowed the model to effectively utilize the data. We finished our data cleaning by scaling the data and removing outliers.

We then started to build and test models, starting with random forest. We tested our models with the training set F1 score and cross-validation. We initially got decent results with our initial random forest model, with an F1 score around 0.85. We then tested both XGBoost and LightGBM and combined them with our random forest model to create a voting classifier. This voting classifier using these three models resulted in an F1 score of 0.89. This result made us believe that a voting classifier would be the best option for this project.

We then moved to hyperparameter tuning and feature engineering to improve our results. We used hyperparameter tuning for the three models, random forest, xgboost, and Lightgbm, in addition to other models that we added to our classifier, including SVM and gradient boosting. Every time we added a new model to the voting classifier or tuned the parameters, our F1 score slightly increased. We then engineered new features to We then engineered new features to capture more meaningful relationships in the data. New features we created included CholesterolPerAge, MaxHRPerAge, and RestingBPPerCholesterol. These features were created by dividing existing features, and as a result, we observed further improvements in cross-validation and F1 scores.

After hyperparameter tuning and feature engineering, we were stuck at an F1 score of around 0.8975. We tried creating more features, but nothing would give us an improved score. We then tried to add more models to the voting classifier, but that did not improve the score either. Finally, changing our voting classifier from a hard vote, where each model casts a single vote for the predicted class, to a soft vote, where predictions are based on the average predicted probabilities, allowed us to reach an F1 score of 0.90196.

Some unique ideas I learned during this project include the importance of understanding the data when creating new features. Many features we tried to add did not add anything of value to our model. When adding features, it's important to find new relationships between existing features that can help us understand the target variable more. In addition, we learned that adding more and more models to a voting classifier comes with diminishing returns. Initially, each model added around 0.015 to the F1 score. Eventually, new models being added resulted in no improvement or the F1 score getting worse.